ACADEMIC PROJECT REPORT

Cinepool Jaipur

Cinema booking and shared rides

Architecture, database engineering and implementation review

Verified against the local source tree, configuration and documentation

and a live MySQL metadata and query-result snapshot.

Source implementation name: Picturehouse Jaipur

Physical database: picturehouse | MySQL 8.4.13

15 base tables | 13 foreign keys | 0 views | 0 triggers

Evidence snapshot: 27 October 2026

Report title follows the requested name; application identifiers are preserved.

FACTUAL SCOPE: implemented, observed and unverified are distinguished

Abstract and evidence statement

Cinepool Jaipur is the requested academic title for the supplied project, whose source code, user interface and package identifiers currently use **Picturehouse Jaipur**. The implementation combines cinema discovery, date-based show selection, a fixed seat map, Clerk email-code authentication, Razorpay Test Mode checkout, persisted QR tickets, optional SMTP delivery, shared ride posts, ride requests and participant chat, and a theatre-owner portal with administrator-approved access.

The inspected configuration defaults to a MySQL schema named **picturehouse**. A connected MySQL Workbench session was used to export metadata and execute eight read-only analytical queries. At 2026-10-27 14:33:36.377226, the server reported MySQL 8.4.13 and exactly **15 base tables, 13 foreign keys, 12 UNIQUE constraints and 15 primary keys**. No stored views, triggers, routines or CHECK constraints were returned. These are observed database facts, not deductions from an older design diagram.

The code implements important transaction and authorization controls, including JWT validation, account-scoped ticket access, approved-theatre checks, payment-signature validation and pessimistic seat locking. These controls are verified as implementation. This report does not claim a complete security audit, production certification, successful external message delivery or exhaustive concurrency validation.

Verification vocabulary. Source verified means a file contains the implementation or declaration. **Runtime verified** means a command or read-only request was executed for this report. **Not verified** means the necessary data, test or provenance evidence was not available. A recommendation is explicitly a future change, not an implemented feature.

Older PRD/SRS/report artifacts describe aspirational features such as seat holds, payment webhooks, WhatsApp delivery and admission-token redemption. Current source takes precedence where those artifacts conflict. The report includes the limitations rather than treating planned features as delivered functionality.

Evidence: [S01-S12], [D1], [T1-T3]. Personal author, roll number, institution and supervisor details were not provided and have not been fabricated.

Contents

Abstract and evidence statement 2

Contents 3

1. Project overview, objectives and scope 4

2. Evidence, provenance and reconciliation 5

3. Actual technology stack and rationale 6

4. Architecture and object-oriented design 8

5. Detailed implemented feature workflows 10

6. Database snapshot and exact inventory 15

7. Physical ER diagram: catalogue, access and legacy models 16

7.1 Exact foreign-key relationship ledger 18

8. Keys, constraints, indexes and data dictionary 19

9. Executed SQL queries and verified results 27

9.1 Triggers, views and subqueries: exact finding 36

10. Normalization analysis with bounded proof 37

11. Transactions, locking and consistency 40

12. Implemented security and its limits 42

13. Verification and testing record 44

14. Limitations, expansion and evaluation 46

15. Conclusion 47

References and development-source provenance 48

Appendix A. Source and evidence index 50

1. Project overview, objectives and scope

1.1 Problem addressed

A cinema visit involves choosing a film, locating a theatre and showtime, buying seats, keeping a ticket and arranging transport. The project places these activities in a Jaipur-focused web application. Theatre partners receive a separate management screen and server-side checks limiting access to approved theatres.

1.2 Objectives supported by implementation

Objective	Implemented mechanism
Discover films and shows	GET /catalog reads active movies and Jaipur theatres with eligible showtimes; customer state filters by date and movie.
Book seats against database inventory	The server calculates fixed seat prices and locks selected inventory rows when confirming a captured test payment.
Retain tickets	ticket_bookings persists a booking reference, payment snapshot, showtime ID and opaque QR credential.
Coordinate shared journeys	ride_posts, ride_join_requests and ride_chat_messages persist ride offers, decisions and chat.
Control theatre management	Clerk JWT subject plus approved membership authorizes owner operations; configured administrator IDs approve requests.
Demonstrate database and OOP concepts	JPA entities, repository interfaces, domain state methods, SQL constraints and transactional controllers provide concrete academic examples.

1.3 Implemented scope and boundary

This is a local prototype with a development catalogue and Test Mode payment integration. Listings are not verified contracts with participating theatres. City labels are fixed to Jaipur. Ride fares are suggested contributions; the code does not charge or settle ride payments. A rendered QR image is implemented; a gate-scanning and one-time-redemption service is absent.

The customer booking window is today plus the next seven dates, inclusive: eight calendar options. The backend uses Asia/Kolkata to filter from its current time until the start of today plus eight days. New owner shows may be created up to 90 days ahead, so some valid owner-created shows are intentionally outside the customer window.

Evidence: App.jsx; CatalogController.getCatalog; OwnerPortalController.addShowtime; RideController. [S01, S03, S06, S07]

2. Evidence, provenance and reconciliation

Evidence	What it establishes	What it does not establish
Source files and dependency manifests	Implemented classes, HTTP routes, validation, frontend flows and configured defaults.	That every branch has executed correctly in a deployed environment.
Live information_schema export [D1]	Exact current tables, columns, keys, indexes and absence of stored objects.	The schema on another machine or after later changes.
Executed SQL Q1-Q8 [D2]	Actual output on the captured local records.	Production inventory, future query output or a complete transaction history.
Build and HTTP observations [T1-T3]	Frontend compilation, Maven completion and selected read-only API outcomes.	Positive authenticated workflows, actual payment/SMTP delivery or concurrency guarantees.
Project documents [S12]	Recorded requirements, guidance and earlier design intentions.	Implementation of a feature that current code does not contain.

2.1 Conflicting documentation resolved

Older statement	Current evidence and treatment
H2 default / PostgreSQL deployment	application.yml defaults to jdbc:mysql://localhost:3306/picturehouse and MySQL driver. PostgreSQL/H2 remain dependencies, not the verified active storage.
Phone and email OTP	ClerkAuth.jsx implements email codes. Phone/SMS OTP is not implemented in this inspected UI.
WebSocket seat broadcasts and temporary holds	WebSocket configuration exists, but the client polls inventory. No hold table, hold API or inventory event publisher was found.
Signed QR, scanner, WhatsApp and signed webhook completion	Current QR contains a UUID ticket_code. No redemption route, WhatsApp adapter or webhook route was found.
Entire database is in 3NF	Several snapshot and comma-separated seat attributes prevent a defensible whole-schema 3NF assertion; Chapter 10 gives bounded proofs.

Report generation added evidence exports and report scripts under tmp/pdfs/cinepool and the final PDF under output/pdf. Application logic and database records were not changed for the report. SQL capture wrote a local evidence file through SELECT ... INTO DUMPFILE; no DDL or data-changing SQL was executed.

3. Actual technology stack and rationale

Version statements below come from package-lock.json, pom.xml, the resolved Maven dependency list and the live server. Rationale is an architectural assessment of observed use. A historical record proving why the original developer chose each tool is **Not verified**; a dated decision log would be required.

Technology / verified version	Observed use and architectural rationale
React / react-dom 19.2.8	Functional JSX components, state and effects implement catalogue, dialogs, seat selection, rides and owner screens. Component composition allows these workflows to share the same page.
JavaScript, JSX, HTML and CSS	The source uses .jsx/.js and a custom stylesheet. No TypeScript, Tailwind or component framework is declared. CSS variables implement light/dark themes.
Vite 8.2.2; React plugin 6.1.0	Development server and production bundling; scripts dev, build, preview and check. The installed versions are locked even where package.json says latest.
@clerk/react 6.17.8	Email-code sign-in/sign-up, session finalization and bearer token retrieval. Outsourcing identity avoids implementing an OTP/password service in this application.
Java 21 target; Spring Boot 3.5.6	Annotated REST controllers, configuration, validation and dependency injection. Java 21 is the configured compilation target, not an independently measured production JVM.
Spring MVC / Tomcat 10.1.46	HTTP routes and an embedded servlet container keep the Java API deployable as a Spring application.
Spring Data JPA 3.5.4; Hibernate 6.6.29.Final	Repositories map Java entities to relational tables, generate routine CRUD SQL and support declared locks. ddl-auto=update evolves the local schema.
Spring Security 6.5.5	OAuth2 resource-server JWT verification, stateless API protection and CORS policy.
MySQL Server 8.4.13; Connector/J 9.4.0	Observed InnoDB persistence, relational constraints and SQL analysis. The JDBC connection integrates with JPA transactions.

3. Technology stack (continued)

Technology / component	Observed use and boundary
Razorpay Standard Checkout	The browser loads checkout.razorpay.com/v1/checkout.js. A custom Spring RestClient calls Orders, Payments and Refund APIs; no Razorpay Java SDK dependency is declared.
ZXing core / javase 3.5.4	QRCodeWriter and MatrixToImageWriter generate a PNG for ticket display and attachment.
Spring Mail / Jakarta Mail	TicketEmailService constructs a multipart HTML message with a QR attachment; configuration requires SMTP and a JWT email claim.
Jakarta Persistence and Validation APIs	@Entity, @ManyToOne, @Column, @Version and request constraints describe persistence and validation rules.
Spring WebSocket / STOMP configuration	An endpoint and simple broker are configured; the inspected frontend does not subscribe and no business publisher was found.
Maven	pom.xml defines the backend build and dependency graph. The local verification used Apache Maven from the existing installation.
PostgreSQL and H2 runtime dependencies	Present in pom.xml; neither is evidence that the active application uses them. The observed schema and current default are MySQL.
Unsplash poster URLs	The UI constructs external poster-image URLs from stored identifiers. These images and external internet access are UI dependencies.
Vercel configuration	A vercel.json exists. Successful public hosting, deployment URL and backend production hosting are Not verified.

3.1 Configuration and runtime separation

The browser API base defaults to http://localhost:8080; Vite defaults to port 5173 with strictPort enabled. The backend sets server.port=8080. Sensitive provider and database settings are read from environment variables. A Vite publishable Clerk key is expected in the frontend; backend Razorpay and SMTP secrets are not returned as application configuration to the browser.

Hibernate schema update is convenient for the prototype but is not a reviewed migration history. Runtime environment variables can override defaults. The observed MySQL server is verified; complete configuration of every external provider is Not verified. This report intentionally excludes credential values and live ticket QR secrets.

Evidence: [S02, S08, S09]; Maven dependency resolution [T2]; live server [D1]. General explanations: [R1-R4, R8-R10].

4. Architecture and object-oriented design

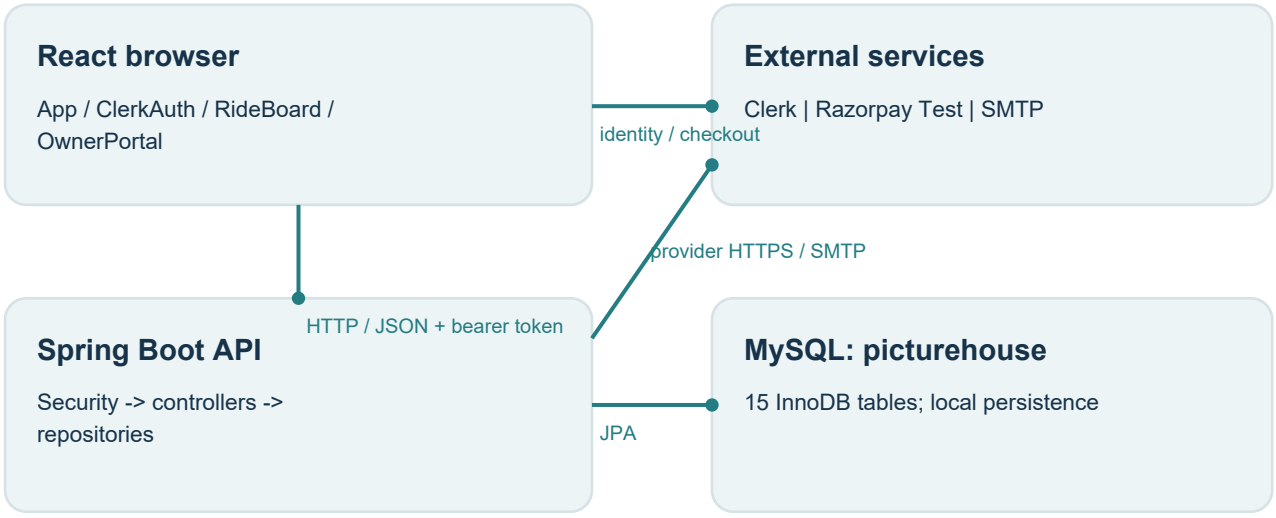


Figure 1. Verified component boundaries. Diagram expresses source dependencies, not an unverified production deployment topology.

The project is a single-page React client and a single Spring Boot backend, with a relational database and external provider integrations. It is not implemented as a set of microservices. Controllers contain substantial business orchestration and transaction boundaries; a fully separated service layer is not present for every feature.

Layer	Responsibilities and exact examples
Presentation	App manages views/dialogs; ClerkAuth manages identity, checkout and tickets; RideBoard and RideChat handle offers/conversations; OwnerPortal renders management forms.
HTTP / business orchestration	CatalogController, RazorpayController, RideController, TicketController and OwnerPortalController validate requests, authorize data access and coordinate persistence.
Persistence	Repository interfaces extend JpaRepository; entity classes map table names and key relationships.
Adapters	RazorpayClient handles provider HTTP calls; TicketEmailService sends SMTP messages; Spring Security obtains Clerk public keys.

4. Object-oriented concepts in the real code

Concept	Concrete implementation	Academic interpretation
Encapsulation	Private entity fields; ShowtimeSeat.markBooked; RidePost.reserveSeat; Booking.confirm/cancel.	Mutation methods express business transitions. Booking is a legacy model, while the active checkout creates TicketBooking.
Abstraction	MovieRepository extends JpaRepository; controllers depend on repository interfaces.	Persistence details are delegated to Spring-generated implementations.
Composition / association	Showtime contains references to Movie and Screen; Screen references Theatre.	The object model represents has-a relationships through JPA associations.
Inheritance	Repository interfaces inherit JpaRepository operations; WebSocketConfig implements WebSocketMessageBrokerConfigurer.	There is no custom domain superclass hierarchy proving elaborate inheritance-based modeling.
Polymorphism	Spring supplies implementations behind repository interfaces; WebSocketConfig overrides interface methods.	Runtime interface dispatch is present. Multiple implemented payment providers are not present.
Constructor overloading	Movie and Theatre define constructors with different argument lists.	One constructor delegates to another to avoid repeating initialization.
Enums and state	RideJoinRequest.Status; RazorpayOrder.Status; Role and BookingStatus.	Named states constrain Java values. The role enum is not the active authorization source.
Dependency injection	Controller and adapter constructor arguments receive repositories/settings/services.	Dependencies are composed by Spring rather than instantiated throughout business code.

Unused abstraction boundary. PaymentGateway and TicketDeliveryGateway interfaces are declared, but no supplied class implements them. RazorpayController directly uses RazorpayClient. Claiming interchangeable payment gateways or a working WhatsApp implementation would exceed the evidence.

Evidence: domain/*.java; repository/*.java; integration/*.java; WebSocketConfig. [S04-S10]

5. Detailed implemented feature workflows

5.1 Discovery, dates and catalogue refresh

On startup App requests /catalog. Current source repeats the fetch every 15 seconds and on window focus or visibility return, preserving the selected movie by ID. Eligible theatres are those returned by the backend with at least one future on-sale show in the eight-date window. Customer date/movie filters are then applied in React. This explains why a three-theatre database can legitimately produce a one-theatre customer response.

The frontend search currently filters movie titles. The search placeholder also mentions theatres, but no equivalent theatre-name filter is implemented in visibleFilms. A film dialog displays up to three matching showtime buttons per theatre using slice(0,3); the theatre section maps all matching shows. The date selector is outside the film modal. Dates are computed once when App mounts, so a page kept open across midnight may retain its old date options.

5.2 Seat selection and pricing

Selecting a show loads /catalog/showtimes/{id}/seats. The seat map polls every 2.5 seconds while open, removes seats that have become booked from the local selection and disables booked buttons. A-F rows and 1-8 columns form a fixed 48-seat layout. Rows A-C cost INR 250, D-E INR 300 and F INR 500. The Java backend independently computes integer paise from seat codes; the browser total is only a display.

A browser selection is not a reservation. There is no hold/expiry mechanism. Another customer can pay for the same available seat during checkout; the final verification step uses locks and a refund attempt to resolve a conflict. Seeded booked flags are demonstration occupancy, not necessarily evidence of a paid booking.

Evidence: App.jsx: loadCatalog, refreshSeats, dateShows and seat-map rendering; CatalogController; RazorpayController.priceForSeatPaise. [S01, S03, S05]

5.3 Checkout and payment confirmation

Step	Actual behavior
1. Require sign-in	The client obtains a Clerk bearer token; Spring validates it. An unsigned request cannot create an order.
2. Create order	POST /api/payments/razorpay/orders accepts a showtime ID and seat codes, not a trusted client price. It checks on-sale status, seat membership, uniqueness, availability and maximum eight selected seats.
3. Price and persist	The server derives titles, theatre and show details from the showtime, calculates paise, calls Razorpay Orders and inserts razorpay_orders with the JWT subject.
4. Open checkout	The browser loads Standard Checkout and sends its returned IDs/signature to /verify. Only test keys are accepted by the backend.
5. Verify provider evidence	The server checks order ownership, HMAC-SHA256 signature, provider order ID, amount, INR currency and captured status returned by Razorpay.
6. Reserve and issue	Inside a transaction, lockSeats obtains write locks. If all requested inventory exists and is unbooked, flags are updated, the order is captured locally and a TicketBooking is saved.
7. Resolve conflict	If inventory is missing/already booked, the code attempts a full refund and records REFUNDED. If that call fails, it returns a support/refund error.
8. Deliver and display	The client attempts the email endpoint separately; whether email succeeds or fails, a successful booking opens My tickets.

An existing booking for the same provider order and user is returned on sequential re-verification. A unique provider_order_id in ticket_bookings helps prevent duplicates. The order row itself is not explicitly locked before that check; concurrent retries and external refund calls require dedicated race tests before a stronger idempotency guarantee can be claimed.

No signed payment webhook endpoint is supplied. Closing the browser before /verify can leave a provider payment awaiting local reconciliation. A database rollback cannot undo a provider-side payment, refund or sent email; this is a multi-system consistency limitation, not a distributed atomic transaction.

Evidence: ClerkAuth.jsx: startPayment and handler; RazorpayController.createOrder/verifyPayment/refundWithoutTicket; RazorpayClient. [S05, S08]

5.4 Ticket history, QR and SMTP email

GET /api/bookings/mine selects TicketBooking rows by the validated Clerk subject and orders by creation time. TicketController encodes the stored ticket_code as a 320 x 320 QR PNG and returns a data URL. The UI renders movie, theatre, show details, selected seats, amount, booking reference and delivery status.

The credential is a random UUID stored in plaintext in ticket_bookings. It is not implemented as a signed JWT or as a one-time consumed gate token. The older Booking.entryTokenHash field exists in a separate model and is not the active TicketBooking QR flow. No scanner or QR validation/consumption endpoint was found.

POST /api/bookings/{reference}/email looks up a ticket belonging to the JWT subject, requires the email claim, and uses TicketEmailService to send HTML plus a QR attachment. Values interpolated into HTML are escaped. A successful send marks email_sent_at. This timestamp records the application accepting the SMTP send call, not proof that the recipient read or even received the message in the inbox.

5.5 Theme, state and display behavior

Light/dark presentation is implemented with CSS variables and React state. Theme persistence across reloads is not implemented in the inspected App source. Film score and poster tags are catalogue display fields; the project does not contain a user-rating or review-submission backend. A new movie can have null poster/score/tag because the owner form does not populate those fields.

Evidence: TicketController.myTickets/emailTicket/qrDataUrl; TicketEmailService; TicketBooking; App and ClerkTicketList. [S01, S08, S09]

5.6 Ride offers, requests and private chat

Stage	Workflow and persistence
Browse	GET /api/rides?date=YYYY-MM-DD is public. It returns active offers for that date ordered by departure; it does not additionally filter already departed offers.
Post	An authenticated user submits pickup, theatre/movie text, show-time text, departure, seats and contribution. The server accepts dates today through +90 days, requires a future departure on that date, 1-6 seats and INR 1-10,000 contribution.
Request	POST /api/rides/{id}/requests rejects the ride owner requesting their own offer and rejects full/inactive offers. One request per ride and requester is enforced by a UNIQUE pair.
Converse	Participants list their conversations. Message reads and writes check that the caller is either ride owner or requester. A declined request closes sending, while history remains readable to participants.
Accept	Only the ride owner can decide a pending request. The code locks both the request and ride, decrements one seat and marks the request ACCEPTED. The offer becomes inactive when seats reach zero.
Decline	Only the ride owner can decline a pending request. Decline changes status without decrementing seats.
Refresh	The chat polls messages every three seconds. The ride list and conversation list reload on relevant UI actions; there is no active WebSocket conversation client.

The active ride model is RidePost, not the older CarpoolRide model. It stores movie_title, theatre_name and show_time as text rather than foreign keys to showtimes. The server therefore does not verify a matching ticket, actual show ID, departure-before-show rule or ownership of a vehicle. The contribution is negotiated outside a payment workflow. In the captured database there is one ride_post, but no join requests or chat rows; those features are source verified, not demonstrated by populated query results.

Evidence: RideBoard.jsx; RideChat.jsx; RideController; RidePost/RideJoinRequest/RideChatMessage; locking repositories. [S06, S10]

5.7 Theatre owner and administrator workflows

Owners use the same Clerk sign-in service as customers. Request access creates or returns theatre_owner_memberships for the subject/theatre pair. Approval is required; a theatre-named Gmail address does not automatically authorize ownership. Administrator permission comes from ADMIN_CLERK_USER_IDS, not from the user selecting a role in the browser or from Account.role.

Operation	Implemented behavior and limits
Owner dashboard	Filters by approved memberships, fetches on-sale shows and groups ticket_bookings by showtime ID. Booking counts count ticket records, not individual seats; gross sales sum amount_paise.
Add movie	Checks approved theatre access; finds an existing movie by case-insensitive title or creates one, then creates a theatre_movies association. Title uniqueness is not enforced by a database UNIQUE constraint.
Edit movie	Requires a theatre_movies link and rejects changes to a movie shared with another theatre. The input accepts a synopsis up to 2,000 characters, but the live column is varchar(255).
Add show	Requires a screen belonging to the approved theatre and an active movie; accepts a future date within 90 days. Inserts the show, theatre/movie link if missing, and exactly 48 unbooked inventory rows in one transaction.
Edit show	Checks theatre ownership and a same-theatre screen, but reschedule changes only time/format. Requested movie/screen changes are not applied. The inspected frontend references editingShow.movieId although ShowSales does not expose that field, which can cause validation failure.
Approve access	GET /api/owner/admin/requests and PUT /api/owner/admin/requests/{id}/approve require a configured administrator subject. Approval changes the membership flag.

The UI currently renders two owner KPI/sales sections. Also, access-request theatre options come from the customer catalogue, which omits theatres with no eligible future shows. These are observable source limitations, not additional owner capabilities. No screen-creation, show cancellation, ticket refund dashboard or theatre identity-proof workflow is implemented.

Evidence: OwnerPortal.jsx; OwnerPortalController; TheatreOwnerMembership. [S07]

6. Database snapshot and exact inventory

Database name: **picturehouse**. Server: **MySQL 8.4.13**. Snapshot: **2026-10-27 14:33:36.377226**, as returned by MySQL NOW(6); session time_zone=SYSTEM. All 15 observed tables use InnoDB and utf8mb4_0900_ai_ci. Counts below are exact COUNT(*) outputs, not approximate information_schema.TABLE_ROWS estimates.

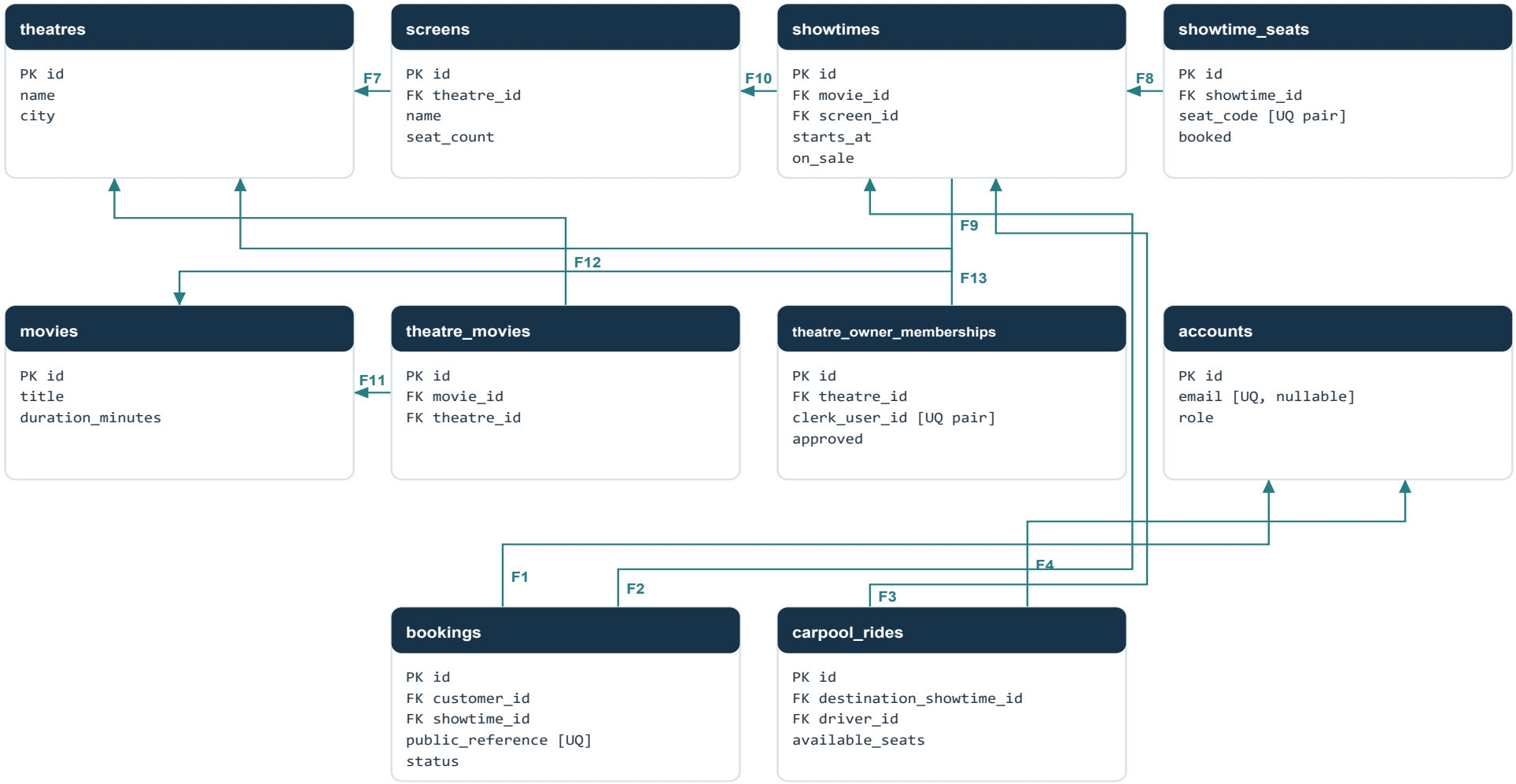
Table	Rows	Implemented role
accounts	0	Legacy local account model; active auth uses Clerk.
bookings	0	Legacy Booking model, not active checkout tickets.
carpool_rides	0	Legacy Account/Showtime-linked ride model.
movies	5	Catalogue movie metadata.
razorpay_orders	3	Provider-order snapshot and payment state.
ride_chat_messages	0	Participant messages for a join request.
ride_join_requests	0	Request/decision for a RidePost.
ride_posts	1	Active ride offers; text movie/theatre snapshots.
screens	3	Physical theatre screens and capacity.
showtime_seats	5856	Per-show seat code and booked flag.
showtimes	122	Movie, screen, start, format and sale flag.
theatre_movies	2	Theatre/movie association used for management.
theatre_owner_memberships	3	Clerk subject to approved theatre access.
theatres	3	Jaipur venues.
ticket_bookings	3	Active paid-ticket snapshot and QR credential.

Snapshot totals include **122 showtimes** and **5,856 seat rows** (122 x 48), but only **3 ticket_bookings**. A booked inventory flag alone must not be counted as a paid ticket because the development seeder deliberately marks example seats booked.

Evidence: [D1]: information_schema.TABLES, COLUMNS, TABLE_CONSTRAINTS, KEY_COLUMN_USAGE, STATISTICS, TRIGGERS, VIEWS, ROUTINES and exact per-table COUNT(*).

7. Physical ER diagram: catalogue, access and legacy models

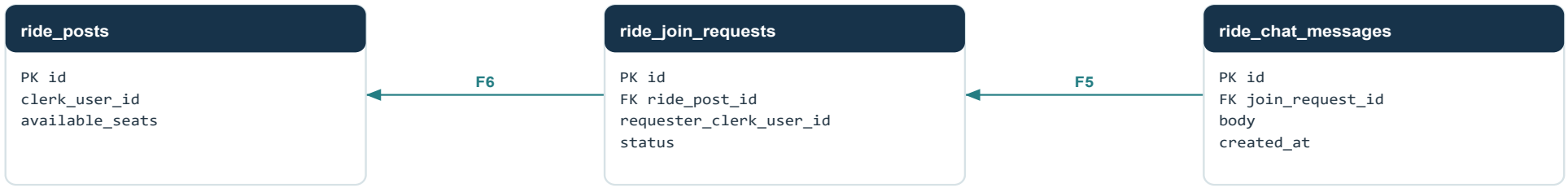
Arrow: child FK -> parent PK (N:1). Each child has one parent; each parent may have zero or many children.



Generated from live information_schema.KEY_COLUMN_USAGE [D1]. Key attributes are shown; the complete column dictionary follows. F labels map to the exact relationship ledger.

7. Physical ER diagram: active rides and payment snapshots

Arrow: child FK -> parent PK (N:1). Each child has one parent; each parent may have zero or many children.



The ride group has two enforced foreign keys. Payment snapshot tables have no enforced foreign keys.

showtime_id and provider_order_id in the payment group are scalar attributes; application joins are logical only.



Generated from live information_schema.KEY_COLUMN_USAGE [D1]. Key attributes are shown; the complete column dictionary follows. F labels map to the exact relationship ledger.

7.1 Exact foreign-key relationship ledger

ID / child	Foreign-key attribute	Parent key
F1 / bookings	customer_id	accounts.id
F2 / bookings	showtime_id	showtimes.id
F3 / carpool_rides	destination_showtime_id	showtimes.id
F4 / carpool_rides	driver_id	accounts.id
F5 / ride_chat_messages	join_request_id	ride_join_requests.id
F6 / ride_join_requests	ride_post_id	ride_posts.id
F7 / screens	theatre_id	theatres.id
F8 / showtime_seats	showtime_id	showtimes.id
F9 / showtimes	movie_id	movies.id
F10 / showtimes	screen_id	screens.id
F11 / theatre_movies	movie_id	movies.id
F12 / theatre_movies	theatre_id	theatres.id
F13 / theatre_owner_memberships	theatre_id	theatres.id

All 13 observed foreign keys have update and delete rules NO ACTION. All child FK columns in this ledger are NOT NULL. No cascading deletion should be claimed. In particular, deleting a referenced movie or theatre may be blocked; the code generally uses active/on-sale fields instead of a complete deletion workflow.

Enforced links versus logical links

ticket_bookings.showtime_id and razorpay_orders.showtime_id are Long scalar fields, not @ManyToOne associations. MySQL reports no foreign keys for either table. The active payment flow nevertheless joins them by showtime ID and associates order/ticket snapshots by provider_order_id. Clerk subject strings identify external accounts without a foreign key to accounts. ride_posts stores theatre/movie names rather than FK references. The ER diagrams intentionally do not draw these as enforced relationships.

Evidence: [D1] and entity mappings. Cardinality follows NOT NULL many-to-one references; zero child rows remain allowed on the parent side.

8. Keys, constraints, indexes and data dictionary

8.1 Primary and unique constraints

Every observed base table has an auto-increment bigint primary key named id. There are 40 declared constraints in the metadata: 15 PRIMARY KEY, 12 UNIQUE and 13 FOREIGN KEY constraints. MySQL reports 38 distinct physical indexes. UNIQUE constraints and foreign-key support indexes are not additional tables.

Table	Unique attribute set
accounts	mobile
accounts	email
bookings	public_reference
razorpay_orders	provider_order_id
ride_join_requests	ride_post_id, requester_clerk_user_id
screens	theatre_id, name
showtime_seats	showtime_id, seat_code
theatre_movies	theatre_id, movie_id
theatre_owner_memberships	clerk_user_id, theatre_id
ticket_bookings	ticket_code
ticket_bookings	booking_reference
ticket_bookings	provider_order_id

A UNIQUE nullable field is not automatically a mandatory candidate key: accounts.email/mobile and screens.name are nullable in the actual schema. MySQL uniqueness permits multiple NULL values. The composite non-null seat, theatre/movie, membership and ride/request pairs provide stronger identity guarantees for those relationships.

8.2 Rules at different layers

The live schema contains no CHECK constraints. Native MySQL ENUM types exist for account role, legacy booking status, Razorpay order status and ride request status. Positive prices, seat limits, future dates and title lengths are chiefly request/domain rules. @Version on Showtime and RidePost adds optimistic version checks for JPA-managed updates; it is not a database trigger.

Explicit ordinary indexes are declared for showtimes.starts_at, ride_posts(ride_date,active) and ride_chat_messages(join_request_id,created_at). The dictionary below reports actual columns, nullability and PK/FK/UQ markings, including all current schema fields. Defaults describe database metadata; Java-side initialization is a separate concept.

Evidence: [D1]; domain and repository source. MySQL behavior reference [R4].

8.3.1 accounts

Legacy local account model; active auth uses Clerk.

Column	Exact MySQL type	NULL?	Observed key / property
id	bigint	NO	PK; auto_increment
display_name	varchar(255)	YES	-
email	varchar(255)	YES	UQ member
email_verified	bit(1)	NO	-
mobile	varchar(255)	YES	UQ member
mobile_verified	bit(1)	NO	-
role	enum('ADMIN','CUSTOMER','THEATRE_OWNER')	YES	-

8.3.2 bookings

Legacy Booking model, not active checkout tickets.

Column	Exact MySQL type	NULL?	Observed key / property
id	bigint	NO	PK; auto_increment
created_at	datetime(6)	YES	-
entry_token_hash	varchar(255)	YES	-
public_reference	varchar(255)	NO	UQ member
status	enum('CANCELLED','CONFIRMED','EXPIRED','PENDING_PAYMENT')	YES	-
total_paise	int	NO	-
customer_id	bigint	NO	FK -> accounts
showtime_id	bigint	NO	FK -> showtimes

8.3.3 carpool_rides

Legacy Account/Showtime-linked ride model.

Column	Exact MySQL type	NULL?	Observed key / property
id	bigint	NO	PK; auto_increment
active	bit(1)	NO	-
available_seats	int	NO	-
contribution_paise	int	NO	-
departure_at	datetime(6)	YES	-
dropoff_area	varchar(255)	YES	-
pickup_area	varchar(255)	YES	-
destination_showtime_id	bigint	NO	FK -> showtimes
driver_id	bigint	NO	FK -> accounts

8.3.4 movies

Catalogue movie metadata.

Column	Exact MySQL type	NULL?	Observed key / property
id	bigint	NO	PK; auto_increment
active	bit(1)	NO	-
certification	varchar(255)	YES	-
duration_minutes	int	NO	-
genre	varchar(255)	YES	-
language	varchar(255)	NO	-
poster_image	varchar(255)	YES	-
score	varchar(255)	YES	-
synopsis	varchar(255)	YES	-
tag	varchar(255)	YES	-
title	varchar(255)	NO	-
tone	varchar(255)	YES	-

8.3.5 razorpay_orders

Provider-order snapshot and payment state.

Column	Exact MySQL type	NULL?	Observed key / property
id	bigint	NO	PK; auto_increment
amount_paise	int	NO	-
clerk_user_id	varchar(255)	NO	-
created_at	datetime(6)	NO	-
movie_title	varchar(255)	NO	-
payment_id	varchar(255)	YES	-
provider_order_id	varchar(255)	NO	UQ member
seats	varchar(255)	NO	-
show_details	varchar(255)	NO	-
showtime_id	bigint	YES	-
status	enum('CAPTURED','CREATED','REFUNDED')	NO	-
theatre_name	varchar(255)	NO	-

8.3.6 ride_chat_messages

Participant messages for a join request.

Column	Exact MySQL type	NULL?	Observed key / property
id	bigint	NO	PK; auto_increment
body	varchar(1200)	NO	-
created_at	datetime(6)	NO	-
sender_clerk_user_id	varchar(255)	NO	-
sender_display_name	varchar(255)	NO	-
join_request_id	bigint	NO	FK -> ride_join_requests

8.3.7 ride_join_requests

Request/decision for a RidePost.

Column	Exact MySQL type	NULL?	Observed key / property
id	bigint	NO	PK; auto_increment
created_at	datetime(6)	NO	-
requester_clerk_user_id	varchar(255)	NO	UQ member
requester_display_name	varchar(255)	NO	-
status	enum('ACCEPTED','DECLINED','PENDING')	NO	-
ride_post_id	bigint	NO	UQ member; FK -> ride_posts

8.3.8 ride_posts

Active ride offers; text movie/theatre snapshots.

Column	Exact MySQL type	NULL?	Observed key / property
id	bigint	NO	PK; auto_increment
active	bit(1)	NO	-
available_seats	int	NO	-
clerk_user_id	varchar(255)	NO	-
contribution_paise	int	NO	-
created_at	datetime(6)	NO	-
departure_at	datetime(6)	NO	-
display_name	varchar(255)	NO	-
movie_title	varchar(255)	NO	-
pickup_area	varchar(255)	NO	-
ride_date	date	NO	-
show_time	varchar(255)	NO	-
theatre_name	varchar(255)	NO	-
version	bigint	NO	-

8.3.9 screens

Physical theatre screens and capacity.

Column	Exact MySQL type	NULL?	Observed key / property
id	bigint	NO	PK; auto_increment
name	varchar(255)	YES	UQ member
seat_count	int	NO	-
theatre_id	bigint	NO	UQ member; FK -> theatres

8.3.10 showtime_seats

Per-show seat code and booked flag.

Column	Exact MySQL type	NULL?	Observed key / property
id	bigint	NO	PK; auto_increment
booked	bit(1)	NO	-
seat_code	varchar(3)	NO	UQ member
showtime_id	bigint	NO	UQ member; FK -> showtimes

8.3.11 showtimes

Movie, screen, start, format and sale flag.

Column	Exact MySQL type	NULL?	Observed key / property
id	bigint	NO	PK; auto_increment
format	varchar(255)	YES	-
on_sale	bit(1)	NO	-
starts_at	datetime(6)	YES	-
version	bigint	NO	-
movie_id	bigint	NO	FK -> movies
screen_id	bigint	NO	FK -> screens

8.3.12 theatre_movies

Theatre/movie association used for management.

Column	Exact MySQL type	NULL?	Observed key / property
id	bigint	NO	PK; auto_increment
movie_id	bigint	NO	UQ member; FK -> movies
theatre_id	bigint	NO	UQ member; FK -> theatres

8.3.13 theatre_owner_memberships

Clerk subject to approved theatre access.

Column	Exact MySQL type	NULL?	Observed key / property
id	bigint	NO	PK; auto_increment
approved	bit(1)	NO	-
clerk_user_id	varchar(255)	NO	UQ member
requested_at	datetime(6)	NO	-
theatre_id	bigint	NO	UQ member; FK -> theatres

8.3.14 theatres

Jaipur venues.

Column	Exact MySQL type	NULL?	Observed key / property
id	bigint	NO	PK; auto_increment
active	bit(1)	NO	-
address	varchar(255)	NO	-
area	varchar(255)	NO	-
city	varchar(255)	YES	-
name	varchar(255)	NO	-
parking_info	varchar(255)	YES	-
theatre_type	varchar(255)	YES	-

8.3.15 ticket_bookings

Active paid-ticket snapshot and QR credential.

Column	Exact MySQL type	NULL?	Observed key / property
id	bigint	NO	PK; auto_increment
amount_paise	int	NO	-
booking_reference	varchar(36)	NO	UQ member
clerk_user_id	varchar(255)	NO	-
created_at	datetime(6)	NO	-
email_sent_at	datetime(6)	YES	-
movie_title	varchar(255)	NO	-
payment_id	varchar(255)	NO	-
provider_order_id	varchar(255)	NO	UQ member
seats	varchar(40)	NO	-
show_details	varchar(255)	NO	-
showtime_id	bigint	NO	-
theatre_name	varchar(255)	NO	-
ticket_code	varchar(36)	NO	UQ member

9. Executed SQL queries and verified results

Queries Q1-Q8 were created for this evaluation and executed against the existing picturehouse records during the snapshot. They are **report queries**, not evidence that the application contains handwritten SQL with those statements. The application principally uses Spring Data derived methods and three declared JPQL locking queries. No nested SELECT was found in those declared JPQL statements or in docs/mysql_teacher_demo.sql.

Every SQL statement qualifies its tables with picturehouse, so it does not depend on Workbench having a default database. Results are time-bound to the captured data. Empty results are reported as empty. Ordering below is normalized for readability by the displayed SQL order; bit values are shown as numeric 0/1, decoded from MySQL JSON binary representation.

Timestamp distinction. Raw SQL Q2 returned 07:00 and 14:30 for shows 121 and 122 on 27 October. The live Java catalogue returned show 122 as **8:00 pm** at INOX Jaipur Central. Do not label raw database DATETIME values as Jaipur wall-clock time. application.yml sets JDBC/Hibernate timezone settings to UTC, while catalogue calculations use Asia/Kolkata. The exact conversion behavior across every driver/session combination is Not verified and requires a timezone round-trip test.

Evidence: [D2] Q1-Q8; public catalogue response [T3]; source [S02-S05].

Q1. Movie names starting with D

Prefix matching

```
SELECT id, title, language, duration_minutes
FROM picturehouse.movies
WHERE title LIKE 'D%'
ORDER BY title, id;
```

id	title	language	duration_minutes
5	DHAMAAL	Hindi	120
2	Dhoom Dhaam Jaipur	Hindi	124

The current collation is utf8mb4_0900_ai_ci. The case-insensitive D prefix matches both DHAMAAL and Dhoom Dhaam Jaipur. This is movie-name data, not a claim that local accounts contains customer names.

Evidence: [D2] Q1; snapshot 2026-10-27 14:33:36.377226. These results can change when the database changes.

Q2. Shows on 27 October 2026: four-table join

Four-table equijoin

```
SELECT s.id AS show_id, t.name AS theatre, m.title AS movie, sc.name AS screen, s.starts_at, s.on_sale
FROM picturehouse.showtimes s
JOIN picturehouse.screens sc ON sc.id=s.screen_id
JOIN picturehouse.theatres t ON t.id=sc.theatre_id
JOIN picturehouse.movies m ON m.id=s.movie_id
WHERE s.starts_at >= '2026-10-27' AND s.starts_at < '2026-10-28'
ORDER BY s.starts_at, t.name, s.id;
```

Show	Theatre / screen	Movie	Raw start / sale
121	PVR Vaishali Nagar / Screen 1	DHAMAAL	2026-10-27 07:00:00 / on_sale=1
122	INOX Jaipur Central / Screen 1	Dhoom Dhaam Jaipur	2026-10-27 14:30:00 / on_sale=1

Two shows are stored on this raw SQL date. on_sale is true for both. The customer catalogue additionally filters from current time, uses its configured date/time semantics and removes theatres with no remaining eligible shows.

Evidence: [D2] Q2; snapshot 2026-10-27 14:33:36.377226. These results can change when the database changes.

Q3. Gross ticket value by theatre, retaining zero sales

LEFT JOIN + GROUP BY + COALESCE

```
SELECT t.id, t.name, COUNT(b.id) AS bookings, COALESCE(SUM(b.amount_paise),0) AS gross_paise
FROM picturehouse.theatres t
LEFT JOIN picturehouse.screens sc ON sc.theatre_id=t.id
LEFT JOIN picturehouse.showtimes s ON s.screen_id=sc.id
LEFT JOIN picturehouse.ticket_bookings b ON b.showtime_id=s.id
GROUP BY t.id,t.name
ORDER BY t.name;
```

id	name	bookings	gross_paise
1	INOX Jaipur Central	1	30000
3	PVR Vaishali Nagar	2	60000
2	Raj Mandir Cinema	0	0

All three theatres remain visible. Three ticket records total 90,000 paise (INR 900). These are stored gross ticket values, not Razorpay settlement reconciliation. COUNT(b.id) counts booking rows, not seats.

Evidence: [D2] Q3; snapshot 2026-10-27 14:33:36.377226. These results can change when the database changes.

Q4. Correlated EXISTS: theatres with an on-sale show from 27 October

Correlated EXISTS subquery

```
SELECT t.id,t.name
FROM picturehouse.theatres t
WHERE EXISTS (SELECT 1
FROM picturehouse.screens sc
JOIN picturehouse.showtimes s ON s.screen_id=sc.id
WHERE sc.theatre_id=t.id AND s.on_sale=1 AND s.starts_at >= '2026-10-27' AND s.starts_at < '2026-11-04')

ORDER BY t.name;
```

id	name
1	INOX Jaipur Central
3	PVR Vaishali Nagar

The inner query refers to the outer theatre ID. It returns only the existence of an eligible raw stored show, without multiplying theatres by their show count. This demonstration uses fixed boundaries for reproducibility, not NOW().

Evidence: [D2] Q4; snapshot 2026-10-27 14:33:36.377226. These results can change when the database changes.

Q5. Scalar subquery: tickets above average booking value

Scalar aggregate subquery

```
SELECT id,movie_title,theatre_name,amount_paise
FROM picturehouse.ticket_bookings
WHERE amount_paise > (SELECT AVG(amount_paise)
FROM picturehouse.ticket_bookings)
ORDER BY amount_paise DESC,id;
```

Verified result: 0 rows. The query executed successfully; no matching records were returned.

The result is empty: no captured ticket row has an amount above the mean in this snapshot. An empty answer is valid and is not replaced with fabricated sample bookings.

Evidence: [D2] Q5; snapshot 2026-10-27 14:33:36.377226. These results can change when the database changes.

Q6. CTE: inventory occupancy without a sales join fan-out

CTE pre-aggregation

```
WITH inventory AS (SELECT showtime_id, COUNT(*) AS total_seats, SUM(booked) AS booked_flags
FROM picturehouse.showtime_seats
GROUP BY showtime_id) SELECT s.id AS show_id,m.title,t.name AS
  theatre,i.total_seats,i.booked_flags,ROUND(100.0*i.booked_flags/NULLIF(i.total_seats,0),2) AS
  flagged_percent
FROM picturehouse.showtimes s
JOIN picturehouse.movies m ON m.id=s.movie_id
JOIN picturehouse.screens sc ON sc.id=s.screen_id
JOIN picturehouse.theatres t ON t.id=sc.theatre_id
JOIN inventory i ON i.showtime_id=s.id
WHERE s.starts_at >= '2026-10-27'
ORDER BY s.starts_at,s.id LIMIT 8;
```

Show / movie	Theatre	Seats / flags	Flagged %
121 / DHAMAAL	PVR Vaishali Nagar	48 / 1	2.08
122 / Dhoom Dhaam Jaipur	INOX Jaipur Central	48 / 0	0.0

Inventory is aggregated before joining catalogue metadata, preventing seat/sales join fan-out. booked_flags measures flags, including seeded demo occupancy where applicable; it is not a paid-ticket count.

Evidence: [D2] Q6; snapshot 2026-10-27 14:33:36.377226. These results can change when the database changes.

Q7. NOT EXISTS: scheduled movies without tickets

EXISTS and NOT EXISTS

```
SELECT m.id,m.title
FROM picturehouse.movies m
WHERE EXISTS (SELECT 1
FROM picturehouse.showtimes s
WHERE s.movie_id=m.id)
AND NOT EXISTS (SELECT 1
FROM picturehouse.showtimes s
JOIN picturehouse.ticket_bookings b ON b.showtime_id=s.id
WHERE s.movie_id=m.id)
ORDER BY m.title;
```

id	title
1	The Last Maharaja
3	The Silent Fort

The Last Maharaja and The Silent Fort have stored schedules but no ticket_bookings joined through their show IDs. This answers the database history question; it does not imply those films have future customer-bookable shows.

Evidence: [D2] Q7; snapshot 2026-10-27 14:33:36.377226. These results can change when the database changes.

Q8. Ride request statuses and persisted message counts

LEFT JOIN grouping

```
SELECT r.id AS request_id,r.status,COUNT(c.id) AS messages
FROM picturehouse.ride_join_requests r
LEFT JOIN picturehouse.ride_chat_messages c ON c.join_request_id=r.id
GROUP BY r.id,r.status
ORDER BY r.id;
```

Verified result: 0 rows. The query executed successfully; no matching records were returned.

The result is empty because there are zero ride_join_requests and zero messages. One existing ride_post does not by itself create a conversation. No chat transcript is invented.

Evidence: [D2] Q8; snapshot 2026-10-27 14:33:36.377226. These results can change when the database changes.

9.1 Triggers, views and subqueries: exact finding

Concept	Verified implementation status	Where it appears
Database triggers	0 observed. No CREATE TRIGGER found in supplied .sql/.java sources.	No verified usage. Seat flags and ride availability change in Java transactions, not in triggers.
Stored database views	0 observed. No CREATE VIEW found in supplied .sql/.java sources.	No verified usage. React screens, DTO records and open-in-view=false are not SQL views.
Stored routines	0 observed procedures/functions in information_schema.ROUTINES.	No verified usage. Java methods are not stored procedures.
Application subqueries	No nested SELECT verified in the three declared JPQL lock queries or teacher SQL demo.	Derived repository SQL is not traced; exact generated SQL plans/subqueries are Not verified.
Report subqueries	Verified by execution for this report.	Q4 correlated EXISTS; Q5 scalar AVG; Q7 EXISTS/NOT EXISTS. Q6 uses a CTE.
Report capture subqueries	Used only to export metadata/results for evidence.	capture_evidence.sql uses scalar aggregate subqueries and JSON aggregation; it is not production application logic.

Potential future uses include an audit trigger for controlled database edits, a view for authorized sales reporting, or subquery-based analytics. Those would require explicit migrations, authorization design and tests. None has been added or presented as an existing feature for this report.

Evidence: [D1] metadata object counts; repository source and docs/mysql_teacher_demo.sql; [D2]. Definitions: [R5-R7].

10. Normalization analysis with bounded proof

10.1 Method and limits of proof

Normalization concerns functional dependencies (FDs), candidate keys and attribute meaning, not just the presence of foreign keys or separate tables. A finite set of rows cannot prove every business FD. The proofs below use declared keys and source semantics; where additional FDs or domain rules are unknown, full normal-form certification is **Not verified**.

```
1NF: one value from the intended atomic domain per attribute.
2NF: 1NF, and no non-prime attribute depends on a proper subset
     of any candidate key.
3NF: for every non-trivial FD X -> A, X is a superkey or A is prime.
BCNF: every determinant of a non-trivial FD is a superkey.
```

A surrogate id does not automatically prove 3NF. It establishes id -> row attributes; other dependencies among non-key attributes must still be considered. Native VARCHAR storage does not make a comma-separated list of domain seat codes normalized for seat-level operations.

10.2 Core decomposition and key dependencies

Relation / determinant	Verified dependency / decomposition	Bounded conclusion
movies.id	id -> title, language, genre, certification, duration and display metadata. No UNIQUE title.	No cross-attribute FD is established by source; 3NF/BCNF compatible under the id-only FD set, not an unconditional proof.
theatres.id	id -> venue attributes. Theatre details are not repeated in screens as FK-linked current data.	Compatible with 3NF under declared dependencies; geography-derived FDs are not assumed.
screens.id	id -> theatre_id, name, seat_count. UNIQUE(theatre_id,name) has nullable name.	The alternate key is conditional on a non-null name; do not assert unconditional candidate-key status.
showtimes.id	id -> movie_id, screen_id, starts_at, format, on_sale, version. Movie and screen details reside in parent tables.	No declared uniqueness of screen/start or collision exclusion; 3NF-compatible for listed dependencies only.
showtime_seats	id and non-null (showtime_id,seat_code) each determine booked.	With FDs K -> all and no further business FD, all determinants are keys: BCNF under that explicitly limited FD set.
theatre_movies	id and non-null (theatre_id,movie_id) determine the row.	The relation is a bridge without additional descriptive attributes: BCNF under its declared-key FD set.

Evidence: Physical keys [D1], entity semantics [S04]. Normalization background [R14].

10.3 Active model exceptions and formal examples

Proof example: showtime_seats

Let R = {id, showtime_id, seat_code, booked} and F = {id -> showtime_id, seat_code, booked; (showtime_id, seat_code) -> id, booked}. Both determinants are candidate keys because id is a primary key and the pair is UNIQUE with NOT NULL columns. Under precisely F, every non-trivial determinant is a superkey, so R is BCNF and hence 3NF. No FD seat_code -> booked is asserted: A1 may be booked in one show and free in another.

Counterexample: repeating seat values

RazorpayOrder stores request.seats with String.join(",", ...), and verification splits the string. TicketBooking copies that seat list. A value such as A1,A2 therefore represents multiple seat-domain values in one attribute. Relative to the intended domain of individual allocated seats, this is a repeating group and is not a 1NF seat-allocation design. Treating the entire string as an opaque snapshot is possible for display, but does not supply relational per-seat keys or constraints.

Counterexample: derived ride_date

RideController.post requires departureAt.toLocalDate() to equal rideDate. In the application-valid subset, departure_at -> ride_date. departure_at is not declared unique and ride_date is not prime. Together with id -> departure_at, this gives a transitive dependency incompatible with 3NF under that source business rule. The rule is validated for that API path, not enforced by a database CHECK.

Snapshots and duplicated identity data

razorpay_orders and ticket_bookings repeat movie_title, theatre_name, show_details, seats and amount across payment/ticket records. These copies support stable display snapshots but introduce redundancy. If showtime_id is interpreted as determining current titles/details, those non-key dependencies violate 3NF. If each copied value intentionally records history at purchase time, that FD requires an explicit historical model and immutability policy, neither fully established here. A whole-schema 3NF assertion is therefore unsupported.

ride_posts repeats movie/theatre/show names without foreign keys; requester/sender display names are repeated in request/message rows. Source does not enforce a single immutable display name per Clerk subject, so a formal subject -> display_name FD cannot be assumed merely because it seems convenient. Account and legacy tables are mostly empty; empty data does not prove their normal form.

10.4 Normalization expansion and migration path

Proposed decomposition	Purpose and key design
users / external identities	Use a local user key plus UNIQUE(provider,external_subject) if local profile governance is needed. Keep Clerk subject mapping explicit.
booking_seats	One row per ticket and showtime-seat allocation, with FK references and a uniqueness rule appropriate to active allocation. Snapshot each seat price as integer paise.
payment_attempts / ticket credentials	Represent provider order/payment identities and ticket issuance separately. Add verified FKs where historic deletion policy allows them.
ride_posts -> showtimes	Reference the actual show rather than uncontrolled theatre/movie text. Derive ride_date from departure or validate a carefully documented materialized copy.
screens / physical seats	Store configurable layouts by screen, then derive per-show inventory instead of always creating A1-F8.
profile versus display snapshot	Separate current user profile data from deliberately preserved historical names; label snapshot semantics and retention policy.

A lossless split of a seat-list booking begins with Booking(reference, purchaser, showtime, amount, ...) and BookingSeat(reference, seat_id, price_paise). Their intersection reference is a key of Booking, so joining on it does not introduce unrelated booking combinations. A migration must parse current comma-separated seats, resolve each code within its showtime, detect duplicate/invalid allocations, preserve historical totals and validate foreign keys before removing the old column.

Separating repeated catalogue details from transactions can reduce update anomalies, but historical ticket wording may intentionally differ from current movie/theatre names. Keep documented snapshot columns where history is required. Formal proof of 3NF for the proposed complete system would require a complete FD catalogue and verified candidate keys after migration.

Evidence: This page is a proposed extension, not a description of tables already present. No new tables were created for the report.

11. Transactions, locking and consistency

Boundary in source	Rows / effect	What is actually protected
RazorpayController.createOrder @Transactional	Creates a provider order and saves razorpay_orders.	Local insert transaction; no seat reservation at this stage. External order creation is outside MySQL atomicity.
RazorpayController.verifyPayment @Transactional	Seat flags, captured order state and ticket insert.	PESSIMISTIC_WRITE seat query serializes conflicting inventory mutations while the transaction holds the lock.
RideController.accept @Transactional	Join-request status and remaining seats on RidePost.	Locks the request and ride, permits only a pending decision by the owner, and commits both changes together.
RideController.decline @Transactional	Pending request -> DECLINED.	A request write lock serializes decisions; no seat decrement occurs.
OwnerPortalController.addShowtime @Transactional	Association link if absent, showtime and 48 inventory rows.	The intended local unit creates the schedule and its inventory together; partial persistence rolls back on ordinary runtime failure.
TicketController.emailTicket @Transactional	Sends email then updates email_sent_at.	Database update is transactional, but SMTP delivery cannot be unsent if the database later fails.
Read-only annotations	Catalogue, ticket history, owner read endpoints, conversation reads.	Declare read-only intent. They do not prove a repeatable report snapshot or security isolation.

11.1 ACID interpretation

Atomicity: related local database writes share a Spring transaction. **Consistency:** NOT NULL, UNIQUE and FK rules work with application validation; missing CHECKs and logical-only references leave gaps. **Isolation:** explicit pessimistic locks protect selected seat/request/ride rows, while Showtime and RidePost also carry optimistic version fields. **Durability:** InnoDB provides persistent storage; crash recovery, backup restoration and actual server durability settings were not tested.

No explicit transaction-isolation level is set in application.yml. The running server isolation level was not captured, so the report does not assert that the application configured SERIALIZABLE or any other level. @Transactional alone is not proof of exactly-once payment completion.

Evidence: Controller annotations and lock repository queries [S05-S10]. General semantics: [R8, R11, R12].

11.2 Runtime SQL versus database trigger behavior

JPA repository save calls and managed entity mutations cause Hibernate-generated INSERT/UPDATE statements. For example, lockedSeats.forEach(ShowtimeSeat::markBooked) mutates managed inventory entities; order.markCaptured updates the managed order; bookings.save inserts a ticket. There is no stored trigger automatically doing this work. Exact generated SQL text was not logged in this review, so an illustrative SQL equivalent must not be mistaken for a captured statement trace.

Reversible classroom transaction example from supplied documentation

```
START TRANSACTION;
INSERT INTO picturehouse.movies
(title, language, genre, certification, duration_minutes, synopsis, active)
VALUES ('SQL Demo - rollback me', 'Hindi', 'Demo', 'U', 1,
        'Temporary classroom row.', 1);
SET @demo_movie_id = LAST_INSERT_ID();
UPDATE picturehouse.movies SET duration_minutes = 2
WHERE id = @demo_movie_id;
ROLLBACK;
```

The supplied docs/mysql_teacher_demo.sql contains this kind of INSERT/UPDATE/ROLLBACK demonstration. It was **not executed for this report**, because the verified query set was read-only. Its presence proves a documented demonstration, not a successful rollback test. AUTO_INCREMENT sequences can still advance for rolled-back inserts; the absence of a practice row does not imply gap-free IDs.

Consistency improvements requiring future work

Add signed webhook reconciliation and an outbox/job mechanism for delivery; put order-state transitions under an explicit concurrency design; test retry/refund races; define booking cancellation and seat release; avoid keeping database locks open during slow provider calls where possible. These are recommendations, not implemented mechanisms.

12. Implemented security and its limits

Control	Verified mechanism	Boundary / qualification
JWT authentication	NimbusJwtDecoder retrieves JWKS, checks signature and default issuer/time validators.	Issuer/JWKS must be configured. A positive real-token verification test was not performed for this report.
Authorized party	Checks azp against configured allowed origins when the claim is present.	Missing azp is accepted by the current validator; it is not a mandatory audience check.
Stateless protected routes	/api/** requires authentication except public GET /api/rides; OPTIONS and /ws/** are permitted.	/catalog is public. /actuator/health is permitted by policy but no actuator dependency/health endpoint is established.
CORS	Exact configured origins, explicit methods/headers, credentials=false.	Allowed localhost origin and rejected unrelated origin were runtime observed. CORS is not an authorization substitute.
Customer isolation	Tickets and payment orders are filtered by JWT subject.	No claim that Account.role controls these endpoints; the active identity is Clerk.
Owner isolation	requireTheatreAccess requires an approved membership for the subject and theatre.	Theatre email naming is not treated as ownership proof. Administrator IDs come from backend configuration.
Chat privacy	participantRequest checks ride owner/requester before message access.	Source verified. A two-user access-denial test is Not verified.
Payment integrity	Server-side prices; order ownership; HMAC-SHA256 with MessageDigest.isEqual; provider captured/amount/currency checks.	Live keys are rejected. A fresh external payment was not made in the report verification.
Request validation	@Valid request records, seat-code patterns, max eight seats, ride/date/range checks.	Database CHECKs are absent; not every entity attribute is guaranteed valid through direct SQL edits.
Output and secrets	React renders text; email escapes HTML; backend settings use environment variables.	No proof of full secret rotation, encryption at rest, CSP, rate limiting or penetration testing.

12. Security observations requiring careful wording

The API disables CSRF and uses stateless bearer tokens. That is a design choice visible in source, not proof that every future cookie-authentication configuration is safe. JWT audience validation is not explicitly configured. QR credentials are UUIDs stored in plaintext; the older hash field in Booking is not the active ticket mechanism. No one-time QR redemption or replay prevention at admission is implemented.

Provider secrets remain server configuration values, while the Razorpay key ID is intentionally returned to the browser. The report did not enumerate, print or reproduce private credential values. .gitignore excludes .env, *.local, node_modules, dist and backend/data. Repository history, previous credential exposure and production secret storage are **Not verified**; assessing them requires an authorized history and secret-management review.

No application rate limiter, CAPTCHA, abuse workflow, ride identity verification, moderation service or payment webhook signature receiver was found. HTTPS is used for Razorpay calls, but the local application defaults use HTTP; production TLS termination and deployment controls are Not verified. JDBC grants, database user privilege minimization, backups and at-rest encryption were not audited.

Claim deliberately not made	Evidence needed
Production secure / penetration tested	Threat model, security test plan, authenticated abuse tests, dependency scan and remediation evidence.
Owner identity verified	Documented onboarding/verification policy and evidence tying an account to its theatre.
Exactly-once charge/booking/refund	Concurrent callback and failure-injection tests, provider reconciliation and durable idempotency state.
QR cannot be copied or reused	Authenticated gate validator, expiry/redemption design, replay tests and atomic consumption record.
Every email arrived	Provider delivery logs and recipient-side confirmation; email_sent_at alone is insufficient.

Evidence: SecurityConfig; RazorpayClient; TicketController; OwnerPortalController; RideController. [S05-S10]. Framework background [R9].

13. Verification and testing record

Checks were performed on 27 October 2026 against the supplied local source tree and already-running API. They were limited to build/dependency inspection and read-only HTTP/database operations. No payments, messages, owner approvals or booking mutations were generated for this report.

Check	Observed outcome	What it proves
npm run build	Passed; Vite 8.2.2 completed production bundling.	The inspected frontend source compiles. It is not proof of correct browser interactions.
Maven test lifecycle	BUILD SUCCESS; testCompile reported No sources to compile.	Backend build lifecycle completed. There is no supplied automated test suite to count as passed.
Maven dependency:list	Completed; versions resolved for Hibernate, MySQL, security, JPA and Tomcat.	Version claims in Chapter 3 match resolution evidence.
Read-only schema capture	Executed through an existing Workbench connection; exported JSON metadata/results.	Counts, keys, column definitions, empty object lists and Q1-Q8 outputs are grounded in live MySQL.
Nine HTTP smoke requests	Results listed below.	Selected public reads and missing-token/CORS boundaries behave as recorded.

GET path	Origin	HTTP	Observed detail
/catalog	(no Origin)	200	5 movies; 1 theatres
/catalog/showtimes/122/seats	(no Origin)	200	48 rows
/catalog/showtimes/999999999/seats	(no Origin)	404	Unknown show rejected
/api/rides?date=2026-10-27	(no Origin)	200	0 rows
/api/auth/me	(no Origin)	401	No bearer token rejected
/api/bookings/mine	(no Origin)	401	No bearer token rejected
/api/owner/dashboard	(no Origin)	401	No bearer token rejected
/catalog	http://localhost:5173	200	5 movies; 1 theatres
/catalog	https://not-allowed.example	403	Origin rejected

HTTP capture time: 2026-10-27T14:36:26.776141+05:30. All nine observations are individual smoke checks; no coverage percentage is claimed.

13.1 Unverified tests and proposed acceptance criteria

Test still required	Expected criterion; not an observed result
Two-user same-seat payment race	At most one ticket allocates the seat; losing captured payment is reconciled and refunded exactly once.
Simultaneous callbacks for one order	A retry returns the same ticket without issuing a refund for the winning payment or creating duplicates.
Ride last-seat concurrency	Two owners actions on separate pending requests cannot make available_seats negative.
Authenticated data isolation	Customer A cannot read/email customer B tickets; nonparticipants cannot read chat; unapproved owners cannot mutate theatre data.
Email-code sign-in and SMTP delivery	Use a real permitted Clerk identity and mailbox; confirm token claims, email attachment and delivery logs.
Date and timezone boundaries	Round-trip owner 20:00 Asia/Kolkata through JDBC, raw SQL, API and browser; test midnight, current-time filtering and today+7 boundaries.
Owner edit and shared metadata	Verify validation and requested field changes, booked-show rescheduling and shared movie isolation.
Recovery	Close browser after provider capture; simulate provider timeout and DB rollback; recover without orphan payments.
Data integrity and backup restore	Exercise uniqueness/FKs and restore a backed-up disposable database; verify migrations reproducibly.
UI / accessibility / load	Run browser acceptance, keyboard/screen-reader checks and measured load tests before performance claims.

No JUnit test sources or test-report XML were supplied in backend/src/test or backend/target/surefire-reports. The POM contains spring-boot-starter-test, but a test dependency is not a test suite. Earlier screenshots show selected user success states; they are historical observations and are not counted here as reproducible end-to-end test passes.

Evidence: [T1-T3]; backend/pom.xml; inspected test directories. No fabricated test cases, pass rates, throughput or latency claims are presented.

14. Limitations, expansion and evaluation

Priority / verified gap	Effect and practical extension
High: payment reconciliation	Browser callback dependency, no webhook and external calls inside local transactions. Add signed provider reconciliation and durable jobs; test failure/retry paths.
High: active-ticket referential integrity	Scalar showtime/order IDs have no database FKs. Introduce explicit references after checking historical rows and deletion policy.
High: QR admission	QR generation has no consumption/expiry scanner. Add authorized gate verification and an atomic redemption record.
High: schedule integrity	No overlap exclusion for shows on the same screen; seeder schedules multiple movies at the same example slot. Add conflict detection with transactional protection.
Medium: fixed inventory/prices	All new shows receive A1-F8, regardless of screen.seat_count. Add screen-specific physical seats and configurable price bands.
Medium: owner editing defects	Show-edit contract mismatch, ignored screen/movie changes and duplicated dashboard rendering. Align DTO/form/API behavior and add workflow tests.
Medium: validation/schema mismatch	Movie synopsis request limit 2,000 versus live varchar(255). Use an explicit schema migration and aligned validation.
Medium: identity/ride denormalization	Clerk IDs are external scalars; ride show details are uncontrolled text; seats are CSV snapshots. Add documented identities and normalized allocations.
Medium: prototype seed/data lifecycle	Seeding runs only when both movie/theatre stores are empty and is disabled by prod profile. It does not roll existing schedules forward daily.
Medium: discoverability/UI	Movie-only search, three-show modal limit, mount-time date list and owner options based on filtered customer catalogue. Separate administration lookup from bookable inventory.
Not verified: deployment quality	Production hosting, TLS, backups, monitoring, real theatre inventory, capacity and disaster recovery require external configuration and operational evidence.

These findings do not justify claiming unused technologies or features as complete. Expansion should start with a versioned migration baseline, a source-controlled test suite and provider reconciliation. Geographic expansion would require configurable cities, theatre contracts, timezone policy and inventory feeds; maps, route matching and automatic ride payment are future proposals only.

15. Conclusion

The supplied implementation demonstrates a coherent local cinema-booking and ride-coordination prototype using React, Java/Spring Boot and MySQL. It contains working public catalogue reads, persistent relational inventory, Test Mode payment confirmation logic, QR ticket generation, account-scoped APIs, owner approval checks and persisted ride conversation models. The live database evidence establishes the exact fifteen-table structure and the analytical SQL outputs recorded in this report.

The strongest academic evidence is the correspondence between concrete source methods, physical constraints, executed read queries and bounded transaction reasoning. The report avoids treating an empty legacy table as an active feature, a view component as a SQL view, a Java mutation as a trigger, or an unused interface as an implemented provider family.

The database is partly normalized, with a clear catalogue structure and constrained bridge/inventory relations, while payment snapshots, comma-separated seat data and ride text fields require explicit qualification. No schema-wide 3NF certification is supported. Production readiness additionally depends on reconciliation, admission validation, schedule integrity, migration discipline and meaningful automated and concurrency tests.

Academic evaluation position: assess the supplied project as an implemented prototype with verified local database evidence and identified engineering gaps. Do not award credit for webhook reconciliation, stored triggers/views, seat holds, WhatsApp delivery, one-time gate redemption or full test coverage unless new source and execution evidence are supplied.

References and development-source provenance

References were checked through web retrieval on 27 October 2026. Only official vendor/project documentation is listed. Package versions are established by local manifests/resolution, not by the latest version displayed on a documentation site. Online documentation can update independently of the project.

A. URLs explicitly linked in supplied project documentation

The links below are recorded in development/setup documents. That establishes a citation in the supplied project, not proof that a particular developer read them. Historical consultation dates, research notes and use during implementation are **Not verified**.

[R1] Razorpay. Standard Checkout integration steps.
https://razorpay.com/docs/payments/payment-gateway/web-integration/standard/integration-steps/

Provenance: docs/RAZORPAY_SETUP.md; payment protocol guidance. Retrieval succeeded (redirects, if any, were followed).

[R2] Clerk. Customize your session token. https://clerk.com/docs/guides/sessions/customize-session-tokens

Provenance: docs/EMAIL_OWNER_SETUP.md; email claim configuration. Retrieval succeeded (redirects, if any, were followed).

[R3] Apache Maven. Windows prerequisites.
https://maven.apache.org/guides/getting-started/windows-prerequisites.html

Provenance: docs/RAZORPAY_SETUP.md; local build-tool setup. Retrieval succeeded (redirects, if any, were followed).

Additional documentation references

B. Additional documentation used to interpret this report

The following pages were consulted for this report. They are not presented as references proven to have been used during original development.

[R4] Oracle / MySQL. MySQL 8.4 CREATE TABLE. https://dev.mysql.com/doc/refman/8.4/en/create-table.html

[R5] Oracle / MySQL. Trigger syntax and examples. https://dev.mysql.com/doc/refman/8.4/en/trigger-syntax.html

[R6] Oracle / MySQL. CREATE VIEW. https://dev.mysql.com/doc/refman/8.4/en/create-view.html

[R7] Oracle / MySQL. Subqueries. https://dev.mysql.com/doc/refman/8.4/en/subqueries.html

[R8] Spring. Spring Framework 6.2: Using @Transactional. https://docs.spring.io/spring-framework/reference/6.2/data-access/transaction/declarative/annotations.html

[R9] Spring. Spring Security 6.5: OAuth 2.0 resource server JWT. https://docs.spring.io/spring-security/reference/6.5/servlet/oauth2/resource-server/jwt.html

[R10] React. Quick Start. https://react.dev/learn

[R11] Spring Data. JPA locking. https://docs.spring.io/spring-data/jpa/reference/jpa/locking.html

[R12] Oracle / MySQL. InnoDB locking reads. https://dev.mysql.com/doc/refman/8.4/en/innodb-locking-reads.html

[R13] Vite. Getting started. https://vite.dev/guide/

[R14] Microsoft Learn. Description of database normalization basics. https://learn.microsoft.com/en-us/previous-versions/troubleshoot/microsoft-365/microsoft-365-apps/access/database-normalization-description

[R15] Eclipse Foundation. Jakarta Persistence 3.1. https://jakarta.ee/specifications/persistence/3.1/

[R16] ZXing project. ZXing repository and documentation. https://github.com/zxing/zxing

[R17] Spring. Spring Boot 3.5 documentation. https://docs.spring.io/spring-boot/3.5/index.html

Appendix A. Source and evidence index

ID	File group	Claim scope
S01	src/App.jsx	Catalogue/date state, refresh and seat map; theme, navigation and booking dialogs.
S02	package.json; package-lock.json; backend/pom.xml; application.yml; vite.config.js	Declared/resolved stack, ports, datasource defaults, schema update and environment variable names.
S03	api/CatalogController.java; repository/ShowtimeRepository.java	Public catalogue filtering, DTOs and seat reads.
S04	domain/*.java	15 entity/table mappings, attributes, constraints, states and associations.
S05	api/RazorpayController.java; repository/ShowtimeSeatRepository.java	Order/verification transactions, server price rules and pessimistic inventory locking.
S06	api/RideController.java; src/RideBoard.jsx; src/RideChat.jsx	Ride validation, participant authorization, acceptance/decline and message polling.
S07	api/OwnerPortalController.java; src/OwnerPortal.jsx	Membership approvals, scoped management and dashboard aggregation.
S08	integration/RazorpayClient.java; src/ClerkAuth.jsx	Provider requests, HMAC verification, email-code UI and checkout callback.
S09	api/TicketController.java; integration/TicketEmailService.java	Ticket access, QR generation and SMTP attachment.
S10	config/SecurityConfig.java; config/WebSocketConfig.java; repository/*.java	Authentication/CORS policy, transport configuration and repository queries/locks.
S11	config/DevelopmentCatalogSeeder.java; .gitignore	Development-only initialization and exclusions.
S12	README.md; backend/README.md; docs/*.md; docs/mysql_teacher_demo.sql; existing PRD/SRS/report DOCX files	Documentation and design intention, reconciled against actual source.

Java shorthand above resolves under backend/src/main/java/in/picturehouse/jaipur/. application.yml resolves under backend/src/main/resources/. Source references are repository-relative so the report remains portable.

Evidence ID	Captured basis
D1	Live metadata and row counts exported to tmp/pdfs/cinepool/evidence.json through MySQL Workbench. Server version, schema and capture timestamp are embedded.
D2	Q1-Q8 query definitions in tmp/pdfs/cinepool/queries.json and results embedded in D1. Queries were authored for this evaluation.
T1	npm run build: successful Vite production bundle, observed in the report session.
T2	Maven test lifecycle and dependency:list: successful completion, no test sources; resolved dependencies retained in maven-dependencies.txt.
T3	Nine read-only HTTP observations in http_checks.json; catalogue JSON retained as catalog_response.json.

Appendix B. Reproducibility and unsupported claims

Evidence fingerprints

The following SHA-256 identifiers bind the report to the inspected local snapshot. The full source manifest records 65 relevant source/configuration/documentation text files. A hash identifies file bytes; it does not establish authorship or code quality.

Database evidence SHA-256:
d32344d34ee3432ae73235dd230c1d430cd1dbdda3884a951c5dea0b2ab9311e

File	SHA-256 prefix (first 16 hex characters)
backend/pom.xml	91461cca8d08ced3
backend/src/main/java/in/picturehouse/jaipur/api/RazorpayController.java	8ce1b02e208a0a73
backend/src/main/java/in/picturehouse/jaipur/config/SecurityConfig.java	b9c870c3e592a440
backend/src/main/resources/application.yml	638265175b057978
package-lock.json	e3eba47c43156d7f
src/App.jsx	d00eb2ec23c162f3
src/ClerkAuth.jsx	6fa7a22f4d5e357b
src/OwnerPortal.jsx	838ec94d7057f689
src/RideBoard.jsx	e7f8a40adacee172

Appendix B. Reproducing and qualifying the evidence

Reproduce key database facts without changing data

```
SELECT COUNT(*) AS base_tables
FROM information_schema.TABLES
WHERE TABLE_SCHEMA='picturehouse' AND TABLE_TYPE='BASE TABLE';

SELECT TABLE_NAME, CONSTRAINT_TYPE, COUNT(*) AS constraints_of_type
FROM information_schema.TABLE_CONSTRAINTS
WHERE CONSTRAINT_SCHEMA='picturehouse'
GROUP BY TABLE_NAME, CONSTRAINT_TYPE;

SELECT TRIGGER_NAME FROM information_schema.TRIGGERS
WHERE TRIGGER_SCHEMA='picturehouse';
SELECT TABLE_NAME FROM information_schema.VIEWS
WHERE TABLE_SCHEMA='picturehouse';
```

Run Q1-Q8 against the same dataset to compare the recorded outputs. Use SHOW CREATE TABLE picturehouse.table_name for an authoritative CREATE definition of one live table. Application SQL INSERT/UPDATE operations are mainly generated by JPA at runtime; this report does not claim an exhaustive historic SQL log or a full database backup.

Not verified	Evidence required to replace the label
Developer research history / authorship	Dated research notes, commit history or decision records tied to actual work.
Global normalization proof	Complete business-domain FD set, candidate-key analysis and explicit snapshot semantics.
External success / production readiness	Provider logs, deployed configuration, actual integration/concurrency/recovery tests and operational review.
Complete historical database contents	A dated authenticated database dump or transaction/binlog evidence; current counts do not reconstruct history.